

VS Code Compatibility Plan: Extensions & Language Intelligence

How Veyra goes from "themes and snippets only" to running the language support and extensions people expect from VS Code, starting with Rust, Go and C++.

Date 7 October 2026 **Baseline** Veyra 0.4.2 **Status** Proposal, awaiting go-ahead

1. Summary

Today Veyra can install only **color themes and snippet** extensions from Open VSX. Every other extension is refused with *"This extension needs capabilities Veyra does not support yet"*, because most VS Code extensions ship program code that runs against VS Code's private extension runtime (the **extension host**), which Veyra does not have.

Full compatibility with every VS Code extension means reproducing that runtime: thousands of APIs that took Microsoft years to build. That is why products such as Cursor and Windsurf are forks of VS Code rather than new editors. Instead of one giant step, this plan delivers what people actually use extensions for, in three phases, each shipped as a normal Veyra update.

Phase	What it unlocks	Relative size
2 Language servers (first)	Real error checking, smart autocomplete, hover docs, go-to-definition, find references, rename and formatting for Rust, Go and C/C++ , then Python, Java, PHP, YAML and more.	Large core, then small per language
1 Declarative extensions	Thousands of Open VSX extensions that contain no code: exact VS Code syntax colours, language packs, file icon themes , language settings and accurate color themes.	Medium-large
3 Extension host	Extensions that run their own code, starting with the most popular (Prettier, ESLint, EditorConfig), then a compatibility layer for a curated list.	Very large, done incrementally

Recommendation

Start with **Phase 2 for Rust, Go and C++**. It delivers the biggest jump in day-to-day quality: red underlines for real errors, completions that understand your types, and navigation across files. These are what the official Rust, Go and C++ VS Code extensions actually provide, and they work by talking to the same open-source language servers Veyra would use. Phase 1 follows, then Phase 3 for selected extensions.

2. Where Veyra is today (0.4.2)

Language	Autocomplete	Error checking
JavaScript / TypeScript	Full IntelliSense across the project	Syntax and type errors
JSON, CSS / SCSS / Less	Yes	Yes
HTML	Tags and attributes	Basic
Rust, Go, C/C++, Python, Java, C#, PHP, Ruby, Kotlin, Swift, shell, SQL	Keywords, snippets, words from open files	None (highlighting only)
Any language	AI ghost-text completions	n/a

Extension type	Status
Color themes	Installs; syntax colours approximated (no TextMate grammars)
Snippets	Installs and works
Language packs, icon themes, language servers, linters, formatters, Git tools, debuggers	Refused at install

3. How VS Code extensions work (and why this is staged)

Declarative contributions

Data inside the extension's `package.json`: themes, snippets, TextMate grammars, language configuration, icon themes, keybindings. No code runs, so Veyra can read and apply these directly. This is **1**.

Extension code

JavaScript that runs in VS Code's Node.js extension host and calls the `vscode` API (commands, editors, workspace, diagnostics, webviews, terminals, debug...). Supporting it means building a compatible host. This is **3**.

Most **language** extensions are thin wrappers around a separate **language server** program that speaks the open Language Server Protocol (LSP). Veyra can talk to those servers directly, without running the extension at all. This is **2**, and it is why Phase 2 gives VS Code-level language support much sooner than a full extension host would.

4. Phase 2: Language servers (Rust, Go, C++ first)

How it works

- Native bridge.** The Rust backend starts each language server as a child process, frames its JSON-RPC messages, and streams them to the editor over Tauri channels (the same mechanism the terminal already uses).

2. **Editor client.** The frontend maps LSP features onto Monaco's providers: diagnostics, completion, hover, signature help, definition, references, rename, document symbols, formatting, code actions and inlay hints.
3. **Document sync.** Open, change and save events are forwarded so the server always sees the current text, including unsaved edits.
4. **One server per language per workspace,** started when a matching file is opened and stopped when the workspace closes.

Servers and setup

Language	Server	How users get it	Notes
Rust	rust-analyzer	<pre>rustup component add rust-analyzer</pre>	Reads <code>Cargo.toml</code> ; formatting via <code>rustfmt</code> .
Go	gopls	<pre>go install golang.org/x/tools/gopls@latest</pre>	Reads <code>go.mod</code> ; <code>gofmt</code> / <code>goimports</code> built in.
C / C++	clangd	Bundled with LLVM (or package manager)	Best with <code>compile_commands.json</code> (CMake: <code>-DCMAKE_EXPORT_COMPILE_COMMANDS=ON</code>).

Veyra detects each server on the PATH (and in common install folders), shows its status in the Health monitor, and when one is missing offers the exact install command. A setting lets users point to a custom server path. Downloading servers automatically is possible later but needs your decision (see section 8).

What users get, per language

- Red/yellow underlines and Problems-panel entries for real compiler errors and warnings, as you type
- Type-aware completions with documentation; auto-imports where the server supports them
- Hover for types and docs; signature help while typing arguments; inlay type hints (Rust, Go)
- F12 go to definition, Shift+F12 references, F2 rename across the project, outline and `@` symbol search
- Format document / format on save; quick fixes (e.g. add missing import, fill match arms)

After Rust, Go and C++

The same bridge then makes each extra language a small addition: Python (Pyright), Java (Eclipse JDT LS, needs a JDK), PHP (Intelephense), YAML and JSON schemas, Dockerfile, Bash, TOML and others.

5. Phase 1: Declarative extensions

- **TextMate grammars** through the same engine VS Code uses (`vscode-textmate` with the Oniguruma regex engine compiled to WebAssembly). Every installed language extension then highlights exactly as in VS Code, and themes become pixel-accurate instead of approximated.
- **Language configuration:** comment toggling, bracket pairs, auto-closing, indentation and folding rules from the extension.
- **File icon themes** (e.g. Material Icon Theme) in the Explorer, tabs and search results.

- **Storage on disk:** extensions move from browser storage (limited to a few MB) to Veyra's app-data folder, removing size limits.
- **Compatibility badges in search:** each Open VSX result shows *Works in Veyra*, *Partly works* or *Needs extension host* before you install, so nothing fails silently.

6. Phase 3: Extension host

Approach	What it means	Assessment
A. Built-in equivalents	Integrate the most-installed tools directly: Prettier and ESLint via the project's own Node install, EditorConfig, Live Server.	Fast, small, reliable. Do first.
B. Compatibility host	A Node.js process implementing the commonly used parts of the <code>vscode</code> API (commands, workspace, configuration, diagnostics, language providers, status bar, messages) for a curated list of extensions.	Large; grows over time. Do second.
C. Adopt an existing host	Embed an open-source host (e.g. from Eclipse Theia or OpenVSCode Server).	Broadest coverage but heavy, and makes Veyra close to a VS Code fork.

Trade-offs to accept for Phase 3B/C

- **Installer size:** bundling a Node.js runtime adds roughly 40–80 MB to today's 7–11 MB installers (it can be an optional download instead).
- **Security:** extensions run third-party code with your permissions; Veyra would show publisher, permissions and a confirmation before first run.
- **Maintenance:** the VS Code API changes monthly; compatibility needs ongoing work.

7. Proposed roadmap

Version	Contents	Done when
0.5.0	LSP bridge, server detection and Health-monitor checks, Rust (rust-analyzer) end to end	Errors, completion, hover, go-to-definition, rename and format work on a real Cargo project
0.5.1	Go (gopls) and C/C++ (clangd); inlay hints; quick fixes	Same checks on Go module and CMake projects on Windows, macOS and Linux
0.6.0	Phase 1: TextMate grammars, language configuration, icon themes, disk storage, compatibility badges	Top 50 Open VSX theme/language/icon extensions install and render correctly
0.7.0	Python, Java, PHP, YAML, Docker servers; Prettier and ESLint built in	Format on save and lint errors on JS/TS projects
0.8.0+	Extension host beta for a curated list	Each listed extension passes its own feature checklist

Sizes and order are planning estimates. Each release keeps the current quality bar: automated tests in the real UI, a release build for all four platforms, signed auto-updates, and release notes in the in-app "What's new".

8. Decisions needed from you

1. **Order:** confirm Phase 2 (Rust → Go → C++) first, or change the order.
2. **Automatic server downloads:** should Veyra download missing language servers itself (more convenient, larger responsibility), or only detect them and show the install command (simpler, the default in this plan)?
3. **Phase 3 size budget:** is a larger installer acceptable, or should the extension host be an optional add-on download?
4. **Test projects:** any real Rust, Go or C++ projects you want used as the acceptance test.

9. Commands you can give me

Say this	And I will
Start Phase 2 with Rust	Build the language-server bridge and Rust support, test it, and ship 0.5.0
Add Go and C++	Add gopls and clangd on the same bridge and ship 0.5.1
Start Phase 1	Build declarative extension support, icon themes and compatibility badges
Make extension <name> work	Prioritise one specific extension or its built-in equivalent
Auto-download language servers	Add verified, opt-in automatic installs of missing servers